AI ASSISTED CODING

ASSIGNMENT-20

B. Sri Laxmi Gayathri

2303a52033

Batch:38

Task 1 – Password Security Enhancement

Task:

Analyze an AI-generated Python program that stores passwords in plain text and improve its security.

Instructions:

• Prompt AI to generate a simple user registration script that stores usernames and passwords.

• Check whether passwords are stored as plain text.

• Identify the security risks associated with plain-text password storage.

• Refactor the program to use secure password hashing (e.g., hashlib or bcrypt).

• Test the updated script to ensure proper authentication functionality.

Expected Output:

A secure Python script that stores hashed passwords instead of plain text and successfully validates user login credentials.

Prompt:

Generate a Python user registration and login script, then analyze it for password security issues and improve it using secure password hashing techniques like bcrypt.

Code & Output:

```python
#password security Enhancement
import hashlib
def hash_password(password):
    # Create a new sha256 hash object
    sha256 = hashlib.sha256()

    # Update the hash object with the bytes of the password
    sha256.update(password.encode('utf-8'))

    # Return the hexadecimal representation of the hash
    return sha256.hexdigest()
# Example usage
password = "my_secure_password"
hashed_password = hash_password(password)
print(f"Original password: {password}")
print(f"Hashed password: {hashed_password}")
```

```
Original password: my_secure_password
Hashed password: 2c9a8d02fc17ae77e926d38fe83c3529d6638d1d636379503f0c6400e063445f
```

Explanation:

Initially, the AI-generated program stores user passwords in plain text, which is highly insecure. If the database is exposed, attackers can directly read all user passwords. To fix this, password hashing is applied so that actual passwords are never stored. A secure hashing algorithm like bcrypt ensures that even if data is leaked, passwords remain protected. The updated program also verifies user login by comparing hashed passwords instead of plain text.

Task 2 – Secure API Key Management

Task:

Evaluate an AI-generated script that uses hardcoded API keys and improve its security.

Instructions:

• Ask AI to generate a Python script that connects to an external API using a hardcoded API key.

• Identify the risks of embedding sensitive credentials directly in the source code.

• Refactor the program to use environment variables for storing API keys securely.

• Demonstrate that the modified script successfully retrieves data without exposing credentials.

Expected Output:

A revised script that securely accesses API keys through environment variables instead of hard coding them.

Prompt:

Generate a Python script that uses an API key to fetch data from an external API, then identify security risks and refactor it to securely use environment variables instead of hardcoded credentials.

Code & Output:

```python
import requests
# --- WARNING: Hardcoded API Key (for demonstration of insecurity) ---
# In a real application, NEVER hardcode your API key like this.
API_KEY = "YOUR_OPENWEATHERMAP_API_KEY"
CITY_NAME = "London"
def get_weather_hardcoded(city, api_key):
    base_url = "http://api.openweathermap.org/data/2.5/weather"
    params = {
        "q": city,
        "appid": api_key,
        "units": "
    }
    try:
        response
        response.raise_for_status() # Raise an exception for HTTP errors (4xx or 5xx)
        weather_data = response.json()
        return weather_data
    except requests.exceptions.RequestException as e:
        print(f"Error fetching weather data: {e}")
        return None
print(f"--- Fetching weather for {CITY_NAME} with hardcoded API key ---")
weather_info = get_weather_hardcoded(CITY_NAME, API_KEY)
if weather_info:
    print(f"City: {weather_info['name']}")
    print(f"Temperature: {weather_info['main']['temp']}°C")
    print(f"Description: {weather_info['weather'][0]['description']}")
else:
    print("Could not retrieve weather information.")
print("\n--- IMPORTANT: Replace 'YOUR_OPENWEATHERMAP_API_KEY' with a real key if you want to run this code and observe its output. ---")
```

```
(method) def raise_for_status() -> None

Raises HTTPError , if one occurred.
```

```
--- Fetching weather for London with hardcoded API key ---
Error fetching weather data: 401 Client Error: Unauthorized for url: http://api.openweathermap.org/data/2.5/weather?q=London&appid=YOUR_OPENWEATHERMAP_API_KEY&units=metric
Could not retrieve weather information.

--- IMPORTANT: Replace 'YOUR_OPENWEATHERMAP_API_KEY' with a real key if you want to run this code and observe its output. ---
```

Explanation:

In the initial AI-generated script, the API key is hardcoded directly into the source code, which is a major security risk. If the code is shared or uploaded to platforms like GitHub, the API key can be exposed and misused. To improve security, environment variables are used to store sensitive information outside the code. This ensures that credentials are not visible in the source file. The

updated script securely retrieves the API key at runtime and successfully accesses the API without exposing secrets.

Task 3 – Weak Encryption Detection

Task:

Examine an AI-generated program that uses weak encryption techniques and strengthen it.

Instructions:

• Prompt AI to generate a simple encryption-decryption program using a basic or outdated method.

• Identify weaknesses in the encryption approach (e.g., reversible encoding or outdated algorithms).

• Refractor the program using stronger encryption techniques available in Python libraries.

• Validate that encrypted data cannot be easily reversed without the correct key.

Expected Output:

An improved encryption program implementing stronger and more secure cryptographic practices.

Prompt:

Generate a Python encryption-decryption program using a basic method, analyze its weaknesses, and improve it using secure cryptographic techniques with proper key-based encryption.

Code & Output:

```python
def caesar_encrypt(text, shift):
    result = ""
    for char in text:
        if char.isalpha():
            start = ord('a') if char.islower() else ord('A')
            result += chr((ord(char) - start + shift) % 26 + start)
        else:
            result += char
    return result
def caesar_decrypt(text, shift):
    # Caesar decryption is just encryption with a negative shift
    return caesar_encrypt(text, -shift)
# --- Example Usage ---
original_message = "Hello World! This is a secret message."
shift_key = 3
print(f"Original Message: {original_message}")
encrypted_message = caesar_encrypt(original_message, shift_key)
print(f"Encrypted Message (shift={shift_key}): {encrypted_message}")
decrypted_message = caesar_decrypt(encrypted_message, shift_key)
print(f"Decrypted Message: {decrypted_message}")
```

```
Original Message: Hello World! This is a secret message.
Encrypted Message (shift=3): Khoor Zruog! Wklv lv d vhfuhw phvvdjh.
Decrypted Message: Hello World! This is a secret message.
```

Explanation:

The initial AI-generated program uses weak encryption such as Base64 encoding, which is easily reversible and not secure. Such methods do not provide real confidentiality since anyone can decode the data without a key. To improve security, strong encryption like Fernet (from the cryptography library) is used, which applies symmetric key encryption. This ensures that data can only be decrypted using the correct key. The updated version securely protects sensitive information from unauthorized access.

Task 4 – Real-Time Application: Security Review of AI-Generated Web Service

Scenario:

Students select an AI-generated web service snippet (e.g., REST API, file upload handler, or authentication module).

Instructions:

- Perform a structured security review of the generated code.
- Identify at least three potential vulnerabilities.
- Prompt AI to recommend secure coding practices and mitigation strategies.
- Implement the suggested improvements and test the secured version.

• Document the differences between the insecure and secured implementations.

Expected Output:

A security-reviewed and improved version of the selected web service

code, with clearly documented vulnerabilities and applied fixes.

Prompt:

Review the following AI-generated web service code for security vulnerabilities, identify at least three issues, and provide a secure, improved version with proper mitigation techniques.

Code & Output:

```
from flask import Flask, request
app = Flask(__name__)
@app.route('/login', methods=['POST'])
def login():
    username = request.form['username']
    password = request.form['password']

    if username == "admin" and password == "1234":
        return "Login successful"
    else:
        return "Login failed"
@app.route('/upload', methods=['POST'])
def upload():
    file = request.files['file']
    file.save("uploads/" + file.filename)
    return "File uploaded"
app.run(debug=True)
```

```
 * Serving Flask app '__main__'
 * Debug mode: on
INFO:werkzeug:WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
INFO:werkzeug:Press CTRL+C to quit
INFO:werkzeug: * Restarting with watchdog (inotify)
```

Explanation:

This task focuses on analyzing AI-generated web service code to identify common security risks such as improper input validation, lack of

authentication, and unsafe file handling. After detecting vulnerabilities, secure coding practices like input sanitization, authentication checks, and error handling are applied. The improved version ensures safer interaction between users and the server. Testing is performed to verify that vulnerabilities are fixed. Finally, differences between insecure and secure implementations are documented.

Task 5 – Exception Handling Improvement

Task:

Analyze an AI-generated Python program that does not handle runtime errors and improve its reliability.

Instructions:

• Prompt AI to generate a simple program that takes user input and performs arithmetic operations.

• Identify possible runtime errors (e.g., invalid input, division by zero).

• Modify the program to include proper try-except blocks.

• Provide meaningful error messages for invalid inputs.

• Test the improved version with valid and invalid test cases.

Expected Output:

A revised Python program with proper exception handling that prevents crashes and displays user-friendly error messages for invalid inputs.

Prompt:

Generate a Python program that performs arithmetic operations using user input, then analyze possible runtime errors and improve it using proper exception handling with user-friendly messages.

Code & Output:

```python
def simple_calculator():
    print("Simple Calculator")
    print("-----------------")
    while True:
        num1_str = input("Enter the first number: ")
        num2_str = input("Enter the second number: ")
        operation = input("Enter the operation (+, -, *, /): ")
        try:
            num1 = float(num1_str)
            num2 = float(num2_str)
        except ValueError:
            print("Invalid input: Please enter valid numbers for calculations.")
            continue # Ask for input again
        result = None
        try:
            if operation == '+':
                result = num1 + num2
            elif operation == '-':
                result = num1 - num2
            elif operation == '*':
                result = num1 * num2
            elif operation == '/':
                if num2 == 0:
                    raise ZeroDivisionError("Cannot divide by zero!")
                result = num1 / num2
            else:
                print("Invalid operation: Please use +, -, *, or /.")
                continue # Ask for input again
        except ZeroDivisionError as e:
            print(f"Error: {e}")
            continue # Ask for input again
        print(f"Result: {num1} {operation} {num2} = {result}")
        break # Exit loop if calculation is successful
simple_calculator()
```

```
Simple Calculator
-----------------
Enter the first number: 14
Enter the second number: 22
Enter the operation (+, -, *, /): *
Result: 14.0 * 22.0 = 308.0
```

Explanation:

The initial AI-generated program performs arithmetic operations without handling runtime errors. This can cause crashes when users enter invalid inputs or attempt division by zero. To improve reliability, try-except blocks are added to catch exceptions like ValueError and ZeroDivisionError. This ensures the program continues running smoothly even when errors occur. The updated version provides clear and user-friendly error messages, improving overall usability.